

SAGMA: SISTEMA DE AGENDAMENTO E GESTÃO PARA PROFISSIONAIS DE MASSOTERAPIA

SAGMA: Scheduling and Management System for Massage Therapy Professionals

Nome completo do autor¹

Nome completo do autor²

RESUMO

Este trabalho propõe o desenvolvimento de uma aplicação multiplataforma (mobile e web responsiva) para suporte à gestão de atendimentos de massoterapia voltada a profissionais autônomos da área de estética corporal. A solução aborda problemas comuns como conflitos de horários, falta de rastreabilidade de pacotes de sessões, dificuldade no controle de clientes, resumo financeiro e materiais consumíveis, atualmente gerenciados por meios manuais ou planilhas eletrônicas. Utilizando o framework Flutter com linguagem Dart e o banco de dados NoSQL Firebase Firestore, a aplicação implementa autenticação diferenciada por perfil, cadastro e gestão de clientes, módulo de agendamento com fluxo de aprovação (estados: solicitado, aprovado, realizado ou cancelado), controle de pacotes com consumo automático, registro de sessões e relatórios básicos. O desenvolvimento seguirá metodologia ágil e iterativa, com prototipagem, implementação incremental e validação com a usuária final (massoterapeuta). Espera-se entregar um MVP (Produto Mínimo Viável) funcional que otimize a rotina da profissional, reduza sobreposições de agenda, melhore o controle de clientes e pacotes e demonstre a viabilidade de soluções digitais personalizadas para pequenos negócios de serviços. A aplicação encontra-se disponível em: <https://github.com/AllandreGirodo/sagma>.

Palavras-chave: agendamento; massoterapia; flutter; firebase; gestão de clientes; aprovação de agenda; aplicação multiplataforma.

ABSTRACT

This work proposes the development of a cross-platform application (mobile and responsive web) to support the management of massage therapy sessions aimed at self-employed professionals in the body aesthetics area. The solution addresses common problems such as schedule conflicts, lack of traceability of session packages, and difficulties in client, financial summary, and material control, currently managed by manual methods or spreadsheets. Using the Flutter framework with Dart language and the NoSQL Firebase Firestore database, the application implements differentiated authentication by profile, client registration and management, a scheduling module with agenda approval flow (states: requested, approved, performed or canceled), package control with automatic consumption, session recording and basic reports. The development will follow an agile and iterative methodology, with prototyping, incremental implementation, and validation with the end user (massage therapist). It is expected to deliver a functional MVP (Minimum Viable Product) that optimizes the professional's routine, reduces schedule overlaps, improves client and package control, and demonstrates the feasibility of personalized digital solutions for small service businesses. The application is available at: <https://github.com/AllandreGirodo/sagma>.

Keywords: scheduling; massage therapy; flutter; firebase; client management; agenda approval; cross-platform application.

¹ Titulação, vínculo institucional e/ou empresarial e e-mail.

² Titulação, vínculo institucional e/ou empresarial e e-mail.

1 INTRODUÇÃO

A transformação digital tem alcançado diversos setores de serviços, incluindo o mercado de estética e massoterapia. O Brasil atingiu uma densidade de dois dispositivos digitais por habitante, com um papel central dos smartphones na rotina de trabalho e entretenimento (MEIRELLES, 2020). Muitos profissionais autônomos ainda utilizam métodos manuais, agendas físicas ou planilhas eletrônicas para agendamentos e controle de pacotes, o que gera sobreposições de horários e ineficiência operacional. O objeto deste projeto é o desenvolvimento de uma aplicação mobile e web responsiva voltada especificamente a profissionais de massoterapia, integrando um fluxo claro de aprovação de agenda e uma gestão de clientes. Diante deste cenário, o presente trabalho busca responder ao seguinte problema de pesquisa: como estruturar e validar um MVP (*Minimum Viable Product*) de sistema de agendamento e gestão que reduza conflitos de agenda por meio de fluxo de aprovação e melhore o controle de clientes e pacotes, com simplicidade de uso e o baixo custo de manutenção?

A justificativa para este estudo reside na carência de ferramentas personalizadas para pequenas clínicas de estética corporal, que frequentemente enfrentam a perda de rastreabilidade de pacotes de sessões e dificuldades no controle de materiais utilizados. A digitalização desses processos pode aumentar a eficiência operacional e reduzir perdas financeiras em pequenos negócios. Como o framework Flutter, e backend gerenciado, como o Firebase, permite reduzir o custo de manutenção e acelerar a entrega de valor através de um MVP, em total alinhamento com os princípios de inovação contínua (RIES, 2012). A aplicação de um fluxo de agendamento com estados e regras de prevenção (solicitado, aprovado, realizado e cancelado) tende a reduzir conflitos e aumentar a confiança na rastreabilidade dos atendimentos prestados.

Para atender a essa necessidade, o objetivo geral deste trabalho é desenvolver e validar um MVP funcional de um sistema de agendamento e gestão de atendimentos de massoterapia utilizando Flutter e Firebase. Como objetivos específicos, o projeto prevê a implementação de autenticação com perfis de acesso diferenciados para profissionais e clientes, além da criação de módulos para cadastro, edição e consulta de clientes. Ademais, será desenvolvido um módulo de agendamento com regras de prevenção de conflitos e um sistema de controle automático de saldo de pacotes de sessões, aplicando a baixa automática imediatamente ao registrar a realização do atendimento.

A estrutura deste trabalho organiza-se da seguinte forma: A Seção 2 fundamenta os conceitos de desenvolvimento multiplataforma, arquitetura *Serverless* e modelagem NoSQL; a Seção 3 detalha o método de pesquisa, abrangendo a infraestrutura em nuvem, engenharia de segurança, requisitos e escopo; a Seção 4 expõe a implementação prática com os resultados visuais das interfaces e ambientes de diagnóstico; a Seção 5 consolida os experimentos computacionais realizados e a análise dos resultados obtidos; a Seção 6 descreve os riscos e ajustes realizados efetivamente; por fim, a Seção 7 apresenta as considerações finais, as limitações identificadas e as propostas para trabalhos futuros.

2 REVISÃO BIBLIOGRÁFICA

Esta seção embasa o desenvolvimento do sistema de agendamento e gestão, abordando conceitos, disciplinas de engenharia de software e decisões arquiteturais empregadas.

2.1 Tecnologias Fundamentais

Na especificação de um sistema, a definição dos limites da aplicação em conjunto com os *stakeholders* determina o escopo do software e sua relação com o ambiente externo (SOMMERVILLE, 2011). Para mapear a distribuição de campos e os vínculos lógicos do banco de dados Cloud Firestore, este trabalho adotou Diagramas de Entidade-Relacionamento (ERD) gerados automaticamente via linguagem de marcação *Mermaid*. Essa abordagem

declarativa, integrada ao repositório no GitHub, assegura simplicidade de manutenção e evolução contínua da arquitetura NoSQL à medida que novas regras de negócio são validadas.

A sustentação dessa estrutura apoia-se em ferramentas modernas alinhadas ao paradigma de desenvolvimento multiplataforma e computação em nuvem, garantindo a manutenibilidade e a escalabilidade da solução. Sob essa perspectiva, a adoção do Flutter fundamenta-se na capacidade de mitigar o endividamento técnico inicial do projeto e acelerar a entrega do MVP, visto que o framework permite desenvolver de forma nativa aplicações para celular (Android e iOS), web e desktop a partir de uma única base de código a custos reduzidos (FLUTTER, 2020), viabilizando a escalabilidade operacional com um ciclo de manutenção unificado. O Quadro 1 sintetiza a matriz tecnológica adotada e sua respectiva aplicação prática no escopo do artefato:

Quadro 1 - Síntese das tecnologias e aplicações no ecossistema do projeto

Tecnologia	Descrição	Aplicação no Projeto
Flutter e Dart	Framework de UI multiplataforma do Google para compilação nativa a partir de uma única base de código.	Desenvolvimento da interface de usuário (UI) para as versões mobile (Android/iOS) e <i>web</i> responsiva, garantindo alta produtividade e experiência consistente.
Firebase	Plataforma de desenvolvimento de aplicativos móveis e web do Google, que oferece diversos serviços backend.	Utilizado para autenticação de usuários (Firebase Authentication) com perfis diferenciados e como banco de dados NoSQL (Cloud Firestore) para armazenamento e sincronização de dados em tempo real.
Cloud Firestore	Banco de dados NoSQL baseado em documentos, parte do Firebase, que oferece sincronização em tempo real e escalabilidade horizontal.	Armazenamento da estrutura de dados do sistema, incluindo coleções de persistência e sincronização de dados operacionais, logs de auditoria jurídica, métricas de produtividade e parametrizações dinâmicas.
Sistemas de Gestão	Conceitos de <i>Customer Relationship Management</i> (CRM) simplificado, ferramentas de agendamento e tratamento de dados estruturados oriundos do Google Agenda para exportação.	Fundamenta a abordagem de gestão de clientes e o fluxo de aprovação de agendamentos, utilizando a exportação de bases locais do Google Agenda em formato CSV para alimentação, higienização e controle operacional automatizado no ecossistema do projeto.
Engenharia de Software	Disciplina que aborda o projeto, desenvolvimento, manutenção e evolução de software.	Aplicação de metodologias ágeis e iterativas, como prototipagem e testes de usabilidade, para garantir que o sistema atenda às necessidades reais da massoterapeuta.

Fonte: Autores.

2.2 Conceitos de Engenharia de Software

O processo de desenvolvimento do sistema adota princípios fundamentais da Engenharia de Software para assegurar a entrega de um produto estável, seguro e alinhado às necessidades reais do ambiente operacional. A Engenharia de Requisitos orienta o levantamento das funções do sistema mapeando desde a esteira operativa de clientes até as dificuldades do agendamento analógico de modo a refletir as regras de negócio da usuária final através de rotinas específicas de validação (SOMMERVILLE, 2018). Complementarmente, a adoção de metodologias ágeis e iterativas viabiliza a construção acelerada de um MVP funcional, permitindo a validação precoce de hipóteses de usabilidade e fluxos de trabalho operacionais (RIES, 2012). Para assegurar a manutenibilidade futura do software e a prontidão da arquitetura para a localização multi-idioma, o projeto incorporou as regras recomendadas de inspeção contínua e análise estática de software (DART, 2026). Essa abordagem orientou a aplicação de ciclos contínuos de refatoração defensiva para padronizar a nomenclatura de componentes, eliminar códigos redundantes, consolidar elementos globais reutilizáveis e estruturar

algoritmos para a higienização de dados brutos vindos de plataformas externas, como o Google Agenda, mitigando de forma ativa o endividamento técnico.

2.3 Modelagem de Dados no Cloud Firestore Orientada a Documentos (NoSQL)

A escolha do modelo não-relacional orientado a documentos para a persistência de dados justifica-se pela flexibilidade de esquema, velocidade de escrita e facilidade de integração nativa com frameworks multiplataforma. Diferentemente dos bancos de dados relacionais tradicionais, que impõem restrições de integridade referencial rígidas e esquemas estáticos, a modelagem no Cloud Firestore organiza as entidades em coleções hierárquicas compostas por documentos estruturados em pares de chave-valor (JSON).

Essa flexibilidade estrutural é indispensável para o tratamento defensivo de dados no ecossistema do projeto, permitindo que o sistema processe e armazene de forma nativa registros com diferentes graus de completude como dados de contatos higienizados e normalizados vindos de arquivos externos do Google Agenda sem a necessidade de migrações complexas de esquema (*migrations*). Adicionalmente, essa abordagem reduz a necessidade de junções (*joins*) em tempo de execução, otimizando o consumo de banda e processamento em dispositivos móveis através de consultas mínimas e reativas que propagam atualizações de estado em tempo real para a interface cliente via *listeners* assíncronos.

2.4 Organização Lógica dos Dados:

Para garantir alto desempenho, isolamento e conformidade estrita à LGPD (Artigo 16 da Lei nº 13.709/2018), as informações no Cloud Firestore foram estruturadas em coleções, documentos e subcoleções NoSQL independentes, mitigando o acoplamento relacional e protegendo registros confidenciais de saúde. A correspondência direta entre as interfaces em Flutter e a real implementação física em nuvem é validada no console de gerenciamento, onde o e-mail normalizado atua como identificador imutável do documento, armazenando atributos cadastrais e isolando subcoleções lógicas em perfeito sincronismo com o *back-end*.

A modelagem lógica e não-relacional da base de dados no *Cloud Firestore* foi estruturada de forma desnormalizada e orientada a documentos através de dez coleções nucleares: *usuarios* (vencendo o mapeamento de credenciais, níveis de acesso e subcoleções aninhadas de anamnese corporal); *agendamentos* (responsável pelo histórico cronológico e persistência dos estados do fluxo de aprovação); *transacoes* (destinada ao controle financeiro de sessões e pacotes); *cupons* (aplicada à gestão de descontos); *configuracoes* (utilizada para parametrizações dinâmicas de preços, travas operacionais e *templates* de mensagens em tempo de execução); *auditoria_credenciais* (dedicada à segurança de acessos); *metricas_diarias* (para consolidação analítica de dados de produtividade); *app_changelog* (voltada à governança pública de versionamento) e *lgpd_logs* (protegida por diretrizes estritas de escrita para o registro imutável e auditável de consentimentos e rotinas de anonimização de dados dos titulares). Essa modelagem distribuída sustenta as regras de negócio do ecossistema por meio das coleções raiz detalhadas em sua documentação em desenvolvimento disponível em: https://github.com/AllandreGirodo/sigma/blob/main/ESTRUTURA_FIRESTORE.md.

3 MÉTODO DE PESQUISA

O desenvolvimento do SAGMA seguiu uma abordagem ágil e incremental baseada nos preceitos do framework Scrum (SCHWABER; SUTHERLAND, 2020), utilizando controle de versão via Git com repositório disponível em: <https://github.com/AllandreGirodo/sigma>.

A evolução das *features* do sistema e a rastreabilidade das implementações foram consolidadas de forma cronológica através de marcos mapeados diretamente do grafo de *commits*, divididos em seis fases consecutivas: a **Fase 0 (Entrevista)** englobou o levantamento

inicial de requisitos e alinhamento de escopo com a profissional; a **Fase 1 (Arquitetura Base)** estabeleceu o setup do repositório e o esqueleto estrutural em Flutter; a **Fase 2 (Core Business e Perfis)** centralizou a modelagem de persistência no Firestore, regras de autenticação e o fluxo operativo de aprovação de agendas; a **Fase 3 (UX e Privacidade)** implementou a localização multi-idioma, o modo escuro e os termos de consentimento digital aderentes à LGPD; a **Fase 4 (DevOps e Automações)** integrou gatilhos assíncronos via Cloud Functions, dashboards analíticos e disparos de notificações; e a **Fase 5 (Deploy)** consolidou a eliminação de débitos técnicos, tratamento global de exceções e a hospedagem final da aplicação.

3.1 Elicitação de Requisitos

Como complemento a essa experiência visual de apresentação, a interface integra botões nativos de redirecionamento para a página profissional da massoterapeuta na rede social Instagram. Essa estratégia de *Deep Linking* permite que o utilizador consulte fotos de antes e veja a profissional como um todo, informações atualizadas e sem atrito de navegação, expandindo o portfólio técnico da clínica para além dos limites estáticos do MVP. O dossiê completo contendo o detalhamento das entrevistas e os critérios de validação disponível em: https://github.com/AllandreGirodo/sagma/blob/main/docs/dossie_entrevistas_elicitacao_requisitos.md. Como resultado inicial deste mapeamento e validação de fluxos, estabeleceu-se a interface visual preliminar do ecossistema voltada à captação e engajamento de utilizadores.

A técnica de elicitação adotada baseou-se em entrevistas semiestruturadas com a esteticista e massoterapeuta Andréia Araújo Campos Girodo, graduada em Estética e Cosmética pela Universidade de Franca (2019-2021), cujo escopo de formação técnico-científica envolve o planejamento, operação e gestão de ambientes de estética (UNIFRAN 2026). Durante as sessões, foram exploradas questões pré-definidas sobre a rotina de atendimentos, anamnese e fluxos de caixa, permitindo ampla compreensão das necessidades reais do negócio sem desviar do escopo do MVP onde foram identificados os requisitos funcionais e não funcionais.

Para assegurar a consistência visual e a portabilidade das interfaces entre diferentes resoluções e sistemas operacionais, o ambiente de desenvolvimento incorporou a ferramenta *Device Preview*. O uso desse componente permitiu simular em tempo real, a partir de uma única execução no navegador web, o comportamento responsivo do SAGMA em múltiplos dispositivos virtuais (como smartphones Android, iOS e tablets). Essa abordagem acelerou a validação de layouts responsivos e a identificação precoce de estouros de pixels (*overflows*) antes da implementação (*deploy*) em produção. Essa validação visual apoia-se diretamente nos recursos de arquitetura das interfaces digitais, cujo design de telas busca mitigar a carga cognitiva do usuário final (BARBOSA; SILVA, 2010). Essa validação visual apoia-se diretamente nos recursos de arquitetura da linguagem Dart (DART, 2026). Para além da análise estática de código (*linter*), o projeto utilizou a tecnologia de compilação dupla do Dart: o mecanismo *Just-In-Time* (JIT), através do recurso *Hot Reload*, viabilizou a renderização e o ajuste imediato de componentes na tela durante as sessões de design; enquanto a compilação *Ahead-Of-Time* (AOT) foi usada para gerar o código nativo de máquina de alta performance para o ambiente final. Adicionalmente, o recurso de *Sound Null Safety* (Segurança de Nulos) do Dart atuou como barreira defensiva no código, garantindo em tempo de compilação que variáveis críticas como identificadores de agendamento e logs de consentimento da LGPD jamais assumissem valores nulos em execução, mitigando falhas críticas no aplicativo.

O SAGMA diferencia-se das plataformas comerciais por unificar a aprovação manual de horários, prontuário de anamnese, conformidade à LGPD, integração social e código aberto de custo zero. A análise comparativa baseou-se nos recursos públicos informados nos canais oficiais das soluções concorrentes (AGENDAPRO, 2026; GETNINJAS, 2026; ICLINIC, 2026; ZENFISIO, 2026). O mapeamento competitivo mostra a necessidade da plataforma customizada sem custos de aquisição. O Quadro 2 consolida essa matriz de mercado:

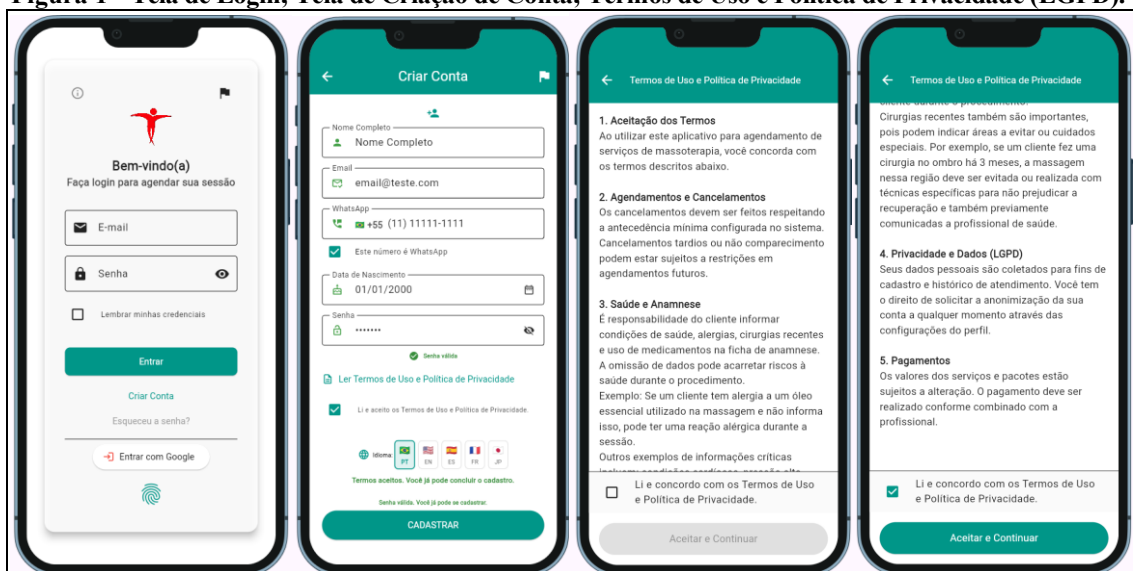
Quadro 2 - Comparativo funcional entre o SAGMA e sistemas de mercado

Critérios / Recursos	SAGMA	ZenFisio	iClinic	AgendaPro	GetNinjas
Multiplataforma (Mobile/Web)	Sim	Sim	Sim	Sim	App
Foco Operacional	Clínica	Clínica	Clínica	Clínica	MarketPlace
Aprovação Manual de Agenda	Sim	Não	Não	Não	Não
Anamnese Integrada	Sim	Sim	Sim	Não	Não
Controle de Pacotes	Sim	Sim	Não	Sim	Não
Conformidade LGPD	Sim	Parcial	Sim	Parcial	Não
Código Aberto (Open Source)	Sim	Não	Não	Não	Não
Suporte Multi-idíomas	Sim	Não	Não	Não	Não
Custo ao Profissional	Gratuito	Pago	Pago	Pago	Comissão

Fonte: Autores.

Superada a análise comparativa, o fluxo de *onboarding* do ecossistema foi projetado para equilibrar segurança jurídica e facilidade de acesso. A Figura 1 ilustra as interfaces de Login, Criação de Conta e a barreira de segurança. O ecossistema suporta autenticação via e-mail/senha, *Google Sign-In* e biometria, exigindo validação de campos em tempo real e o aceite mandatório dos Termos de Uso e Política de Privacidade alinhados à LGPD.

Figura 1 - Tela de Login; Tela de Criação de Conta; Termos de Uso e Política de Privacidade (LGPD).



Fonte: Autores.

Os Termos de Uso acoplados a esta etapa apresentam cinco cláusulas restritas (1. Aceitação dos Termos, 2. Agendamentos e Cancelamentos, 3. Saúde e Anamnese, 4. Privacidade e Dados/LGPD e 5. Pagamentos), cujas assinaturas digitais são persistidas de forma imutável no Cloud Firestore. Imediatamente após a submissão do formulário, o sistema redireciona o fluxo operativo para a interface de (Aguardando Aprovação) informando que o cadastro está em análise e dispara um gatilho automatizado que notifica a profissional administradora via WhatsApp. A notificação consiste na mensagem pré-formatada gerada a partir da integração com os barramentos de mensageria da plataforma (META, 2026), contendo nome, telefone, e-mail e a data/hora (*timestamp*) exato do registro para triagem.

3.2 Infraestrutura e Atalhos de Serviços no Console Firebase

Sobre a infraestrutura de nuvem do Firebase, operando sob o conceito de *Backend as a Service* (BaaS), o console administrativo concentra três módulos integrados de suporte para a governança e distribuição do software. O *Authentication* gerencia identidades e controle de acessos de ponta a ponta (RF01), isolando credenciais e emitindo *tokens* seguros com identificadores exclusivos (uid) que vinculam as interfaces em Flutter aos perfis de "profissional" ou "cliente". O *App Check* atua como mecanismo de segurança cibernética,

atestando a autenticidade do dispositivo e do binário do SAGMA (móvel ou *web*), bloqueando requisições ilegítimas e prevenindo custos imedidos. Por fim, o *App Hosting* centraliza o DevOps para a automação do *pipeline* de integração e implantação contínuas (CI/CD), monitorando os *commits* no GitHub para compilar e distribuir o código Dart/Flutter em servidores de borda de alta disponibilidade com SSL automático.

Para garantir a eficiência de memória, a camada de serviços utiliza o padrão *Lazy Singleton*, que posterga a alocação de recursos densos até o momento de sua primeira requisição (DART, 2026). Esta estratégia garante que classes críticas como o gerenciador de persistência do *Cloud Firestore* e o módulo de *Authentication* possuam uma única instância ativa na memória do dispositivo, sendo alocadas estritamente sob demanda para otimizar o tempo de inicialização (*cold start*) do aplicativo e preservar uma conexão reativa estável com o Firebase.

O mapeamento visual que orienta a árvore lógica de decisão encontra-se documentado em repositório cuja especificação estendida está disponível em: <https://github.com/AllandreGirodo/sagma/blob/main/docs/diagramas/fluxograma.md>. No fluxograma consolida os principais fluxos de autenticação, gestão de clientes, agendamento e integrações com o banco de dados, explicitando como o sistema aplica validações e atualizações de estado ao longo da operação.

3.3 Desenvolvimento, Engenharia de Requisitos, Dinâmica Operacional e Segurança

O ciclo dos agendamentos para atendimentos é regido por um fluxo reativo de aprovação de agenda, iniciado quando o cliente gera um registro com estado "solicitado" (RF03). A profissional realiza a operativa, detendo privilégios exclusivos baseados em seu perfil (*role == 'profissional'*) para alterar o estado para "aprovado" ou "cancelado" (RF04). Ao modificar o status para "realizado", o sistema dispara uma rotina síncrona de baixa automática no saldo do pacote ativo, decrementando a variável de controle (*remainingSessions--*) (RF05).

Algoritmos defensivos barram conflitos de horários concorrentes para a mesma profissional (RNF04 - Confiabilidade) e mantêm o histórico de mutações persistido para auditoria. O módulo de gestão cadastral centraliza as operações de inclusão, edição e consultas históricas de clientes (RF02), estabelecendo o vínculo automatizado entre usuário, pacotes e sessões, com suporte a notificações e solicitações de alterações via WhatsApp.

A governança de acessos é imposta por Controle de Acesso Baseado em Perfis (RBAC) via regras declarativas do servidor (*Firestore Security Rules*) (RNF01). Sob esta diretriz, a profissional detém privilégios irrestritos de leitura e escrita sobre as coleções, ao passo que os clientes limitam-se a acessar seus próprios registros através de validações determinísticas de identidade (*request.auth.uid == userId*). Atualizações no campo de *status* e ações sensíveis são bloqueadas na camada do cliente e restritas à profissional, mitigando fraudes, garantindo a segregação de permissões e atendendo aos critérios de conformidade com a LGPD (RNF05) por meio da minimização de dados e suporte à exclusão definitiva.

O desenvolvimento foi organizado em etapas progressivas, iniciando com a entrega do anteprojeto, levantamento bibliográfico, detalhamento metodológico e configuração do ambiente. Em seguida, implementou-se o módulo de autenticação (RF01) e a estrutura inicial do banco de dados no *Firebase Firestore*. Na sequência, desenvolveram-se as interfaces principais (RNF02 - Usabilidade), a gestão de clientes, o agendamento com fluxo de aprovação e a integração inicial com mensagens customizáveis no *WhatsApp*. Por fim, foram implantados o controle transacional de pacotes com consumo automático, o registro de sessões concluídas, a geração de relatórios estatísticos em formato de listagem com filtros temporais (RF06, RNF03 - Performance) e os testes com a massoterapeuta. A entrega mínima está estritamente a estas funcionalidades operacionais nucleares. Foram deliberadamente excluídas do escopo desta pesquisa rotinas de integração com *gateways* de pagamento assíncronos, emissão de notas fiscais eletrônicas e algoritmos de otimização avançada baseados em Pesquisa Operacional, elementos catalogados como propostas para trabalhos futuros.

3.4 Plano de Validação

Os testes de usabilidade do SAGMA serão conduzidos com utilizadores reais para avaliar a eficácia do ecossistema em tarefas críticas (cadastrar cliente, criar agendamento, aprovar sessão, registrar realização e consultar relatórios). O objetivo é validar, por meio de métricas como a satisfação do utilizador e uma taxa de sucesso esperada de $\geq 80\%$ índice de corte referenciado nos modelos de eficácia da norma ISO/IEC 9241-11 e nos padrões empíricos de mercado estabelecidos por Nielsen (2000), se a interface desenvolvida em Flutter é intuitiva, mitigando erros operacionais e eliminando barreiras cognitivas na rotina da clínica.

4 MODELAGEM DO SISTEMA

O painel operativo do ecossistema distribui-se em interfaces responsivas projetadas especificamente para atender aos perfis de acesso mapeados, garantindo consistência tanto em dispositivos móveis quanto em terminais *web*. A portabilidade da solução expande o *layout* de tabelas e grades de horários automaticamente em telas maiores através da responsividade nativa do *framework* Flutter. O ambiente restrito da administradora centraliza o gerenciamento operativo em quatro abas principais: indicadores analíticos (*Dashboard*), controle de agendas, cadastro de clientes e triagem de solicitações pendentes de aprovação. O *Dashboard* consolida a volumetria de atendimentos, estimativas de faturamento mensal, *status* dos serviços e taxas de absenteísmo por período, cujos *snapshots* históricos de produtividade são persistidos de forma reativa na coleção *metricas_diarias* para suporte à tomada de decisão. Como complemento à gestão visual, o ecossistema realiza a sincronização unidirecional de estados com a *Google Calendar API*, permitindo que as sessões validadas no aplicativo constem na grade cronológica externa da profissional, mitigando conflitos de concorrência.

A gestão de clientes indexa registros por nome ou e-mail, vinculando o perfil ao saldo de sessões e ao histórico de pacotes. Para otimizar a experiência do usuário (UX), o sistema suporta localização em cinco idiomas (português, inglês, espanhol, francês e japonês) e gerenciamento dinâmico de temas (incluindo dark mode modo escuro) via *Cloud Firestore*. O detalhamento das interfaces e o manual de operação constam no repositório disponível em: https://github.com/AllandreGirodo/sagma/blob/main/docs/MANUAL_ADMINISTRATIVO.md.

A interface do cliente prioriza a usabilidade com fluxos diretos para demandas operacionais. A tela principal exibe o histórico cronológico de sessões, busca textual e um botão de ação flutuante (*Floating Action Button*) para novas requisições. O fluxo de marcação orienta o usuário na seleção do procedimento (com marcador de favoritismo), listagem de horários disponíveis calculados defensivamente contra conflitos e aplicação de cupons ativos. Após a confirmação, o registro é inserido no banco como "solicitado", disparando uma notificação automatizada via WhatsApp com os metadados da agenda e identificadores exclusivos de controle como *log* comparativo de segurança. O portfólio de telas e diagramas de navegação constam na documentação suplementar indexada no repositório do projeto como artefatos suplementares de engenharia e disponíveis em: https://github.com/AllandreGirodo/sagma/blob/main/docs/MANUAL_CLIENTE.md.

4.1 Ambiente de Diagnóstico, Governança e Customização Visual

A integridade do ecossistema e as ferramentas de suporte ao desenvolvedor foram isoladas sob autenticação de dois fatores (2FA) e variáveis de ambiente confidenciais. O painel de desenvolvimento (*DevTools*) integra o módulo *DB Manager*, que monitora a volumetria de documentos NoSQL em tempo real e fornece rotinas controladas para a visualização, inserção e exportação de dados em formatos estruturados (JSON, CSV e Excel) para auditoria administrativa. A governança pública do software é realizada por meio de uma interface interna de *changelog*, notificando sobre atualizações e evoluções de versão nas interfaces *mobile* e *web*.

Durante o ciclo de engenharia, a documentação de barramentos lógicos e modelos de dados foi padronizada sob a especificação OpenAPI 3.0 (*Swagger*), cujo contrato de interface encontra-se estruturado em arquivo YAML e disponibilizado via servidor local (*localhost*) na porta 8080, permitindo o mapeamento estrito das estruturas de objetos JSON que trafegam na camada de persistência. Os testes de persistência, validação de regras de escrita e leitura (*Security Rules*) e o monitoramento de requisições HTTP foram conduzidos de forma isolada via *Firebase Emulator Suite*, garantindo o comportamento defensivo do banco de dados antes do *deploy* final para produção, em alinhamento com as diretrizes internacionais de segurança da informação (ISO/IEC, 2022). Essa camada de controle é essencial para a integridade de registros em sistemas de informação que operam em nuvem (CANNIATTI, 2012).

Para otimizar a experiência do usuário (UX), a interface administrativa permite a localização ágil de registros por indexação de nome ou e-mail, acessando dados de saldo de sessões, pagamentos e pacotes vigentes. O sistema incorpora suporte nativo à localização multinacional em cinco idiomas (inglês, espanhol, francês, japonês e o português brasileiro como padrão) e gerenciamento dinâmico de temas visuais (incluindo o modo escuro) a partir de chaves de configuração distribuídas diretamente pelo *Cloud Firestore*. Os esquemas OpenAPI e os relatórios gerados pelo Emulator Suite encontra em: https://github.com/AllandreGirodo/sagma/blob/main/docs/DEVOPS_AND_EMULATORS.md.

5 RESULTADOS ESPERADOS

Os resultados práticos do ecossistema SAGMA consolidam-se na entrega de um ambiente multiplataforma funcional e adaptado às rotinas operacionais da clínica de estética. A aplicação *web* responsiva foi implantada via infraestrutura global do *Firebase Hosting*, disponibilizando o acesso homologado em ambiente de produção através da URL hospedado em: <https://horario-agenda.web.app/sagma> (GIRODO, 2026). O MVP entregue materializa-se na unificação sistêmica de quatro capacidades nucleares: a gestão de clientes (abrangendo a inclusão, edição e consultas a históricos de atendimentos); o agendamento otimizado (composto por um fluxo reativo de estados de aprovação e algoritmos defensivos para prevenção de conflitos de horários); controle transacional de pacotes (responsável pelo decremento automatizado de sessões); e a emissão de relatórios gerenciais básicos estruturados em listagens e filtros temporais para acompanhamento financeiro e de produtividade. Esta solução digital otimiza a rotina da profissional de massoterapia, reduzindo a carga administrativa e minimizando erros operacionais. Adicionalmente, promoveu uma melhoria significativa na experiência da cliente por meio de agendamentos transparentes e comunicação eficiente servindo de prova de conceito da viabilidade de sistemas digitais personalizados para pequenos negócios na área de estética, consolidando o potencial para futuras expansões, tais como a implementação de serviços automatizados de mensageria para lembretes e a integração de visões de calendário estendidas para clientes e profissionais.

6 RISCOS E MITIGAÇÕES

A gestão de riscos é um componente crucial no desenvolvimento de projetos de software, permitindo a identificação proativa de potenciais problemas e a formulação de estratégias para mitigá-los (PRESSMAN, 2016). No contexto de arquiteturas *Serverless*, a antecipação de falhas operacionais e vulnerabilidades lógicas preserva a integridade dos dados, a segurança cibernética e a continuidade estável do negócio. Esta abordagem preventiva alinha de forma estratégica e transparente as expectativas da engenharia com as restrições reais de tempo e recursos essenciais de um Produto Mínimo Viável. No projeto, os fatores críticos de sucesso foram monitorados associando a probabilidade de ocorrência ao impacto gerado na clínica de massoterapia. No Quadro 3, encontram-se destacados os seguintes riscos principais identificados e suas respectivas estratégias de mitigação aplicadas ao longo do ciclo de vida do desenvolvimento do SAGMA:

Quadro 3 - Matriz de riscos do projeto e respectivas estratégias de mitigação

Risco	Descrição	Estratégia de Mitigação
R1 - Escopo Amplo Demais	A tendência de expandir as funcionalidades do sistema além do planejado para o MVP, o que pode atrasar a entrega e qualidade.	Congelamento do escopo do MVP tendo definição clara e rigorosa das funcionalidades essenciais para a primeira versão do produto. Funcionalidades adicionais serão documentadas e consideradas para futuras iterações do projeto.
R2 - Falta de Usuários para Teste	Dificuldade em recrutar usuários que são reais (massoterapeuta e clientes) para a fase de testes e validação, impactando a qualidade e a usabilidade.	Confirmação antecipada de participantes estabelecimento de acordos formais com a massoterapeuta e potenciais clientes para garantir sua participação nos testes. Criação de um cronograma flexível para acomodar a disponibilidade dos testadores.
R3 - Dificuldade com Regras de Segurança do Firestore	Complexidade no (<i>deploy</i>) e configuração das regras de segurança do <i>firestore.rules</i> , que levam vulnerabilidades.	Implementação de regras mínimas no início com um conjunto básico de regras de segurança, expandindo-as gradualmente. Realização de revisões em segurança Firebase para garantir a robustez e a conformidade.
R4 - Curva de Aprendizagem de Novas Tecnologias	A necessidade de dominar tecnologias como Flutter, Dart e Firebase pode prolongar o cronograma e introduzir erros.	Capacitação contínua e documentação a dedicação de tempo para estudo e prática das tecnologias envolvidas. Utilização de documentação oficial, tutoriais e comunidades de desenvolvedores para acelerar o aprendizado e resolver problemas.
R5 - Integração com WhatsApp	Desafios técnicos na integração do sistema com a API do WhatsApp para envio de notificações e solicitações de alteração.	Ferramentas e bibliotecas existentes de pesquisa e utilização de bibliotecas ou serviços de terceiros que facilitem a integração com o WhatsApp. Focado em funcionalidades básicas de notificação e escalonamento para recursos mais complexos.

Fonte: Autores.

7 CONSIDERAÇÕES FINAIS

O desenvolvimento do ecossistema SAGMA permitiu a entrega de uma solução multiplataforma funcional que mitigou os principais gargalos operacionais enfrentados por profissionais autônomos de massoterapia. Ao centralizar a gestão de clientes, agendamentos e pacotes contratuais, validou-se a hipótese de que a digitalização de processos em pequenos negócios reduz drasticamente a ocorrência de conflitos de agendas e a perda de informações financeiras e de consumo de insumos. O projeto demonstrou que a escolha do framework Flutter com a linguagem Dart, combinada à infraestrutura *Serverless* do Firebase, foi determinante para a agilidade do ciclo de desenvolvimento e para a entrega de interfaces fluidas e responsivas nos ambientes mobile e web. A implementação do fluxo reativo de estados trouxe rastreabilidade ao controle de pacotes de sessões, enquanto a baixa automática de saldo consolidou-se como um diferencial crítico para a eficiência operativa da profissional.

Contudo, foram mapeadas limitações técnicas e operacionais que delineiam oportunidades para o refinamento do artefato. No escopo da comunicação, a automação de notificações permaneceu restrita à geração de hiperlinks dinâmicos direcionados ao WhatsApp, deixando pendente a integração com uma API oficial de mensageria, o disparo de alertas *push* ou o acoplamento de fluxos automatizados de atendimento via plataforma n8n (N8N, 2026). Na camada de análise gerencial, o sistema limitou-se à exibição de listagens e filtros textuais, carecendo de dashboards gráficos interativos para a avaliação sintética do faturamento.

Para as próximas iterações evolutivas do *software*, planeja-se a integração com a *Google Calendar* API para sincronização externa voltada aos clientes, a implementação de gateways de pagamento assíncronos para a reserva antecipada de horários com liquidação financeira integrada, a automação da emissão de documentos fiscais e a aplicação de algoritmos de Pesquisa Operacional para a otimização inteligente da grade horária da profissional.

REFERÊNCIAS

- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6028**: informação e documentação - resumo: apresentação. Rio de Janeiro: ABNT, 2021.
- BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Dispõe sobre a Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, [2018]. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 28 maio 2026.
- BARBOSA, S. D. J.; SILVA, B. S. **Interação humano-computador**. Rio de Janeiro: Elsevier, 2010.
- CANNIATTI, A. C. B. **Auditoria de sistemas de informação**. São Paulo: Editora Sol, 2012.
- DART. **Lint Rules**. Disponível em: <https://dart.dev/tools/linter-rules>. Acesso em: 28 maio 2026.
- DART. **Effective Dart: Style**. Disponível em: <https://dart.dev/guides/language/effective-dart/style>. Acesso em: 28 maio 2026.
- FLUTTER. **Flutter Documentation**. Disponível em: <https://docs.flutter.dev>. Acesso em: 30 mar. 2026.
- SAGMA. **Repositório do ecossistema SAGMA**: documentação, diagramas e código-fonte. GitHub, 2026. Disponível em: <https://github.com/AllandreGirodo/sagma>. Acesso em: 28 maio 2026.
- GIRODO, Allandre Ramos. **SAGMA: Agendamento Massoterapia**. Firebase Hosting, 2026. Disponível em: <https://horario-agenda.web.app/sagma>. Acesso em: 28 maio 2026.
- GIT. **Git: fast version control system**. Versão 2.40. Washington: Software Freedom Conservancy, 2026. Disponível em: <https://git-scm.com>. Acesso em: 28 maio 2026.
- GOOGLE. **Firebase Documentation**. Disponível em: <https://firebase.google.com/docs>. Acesso em: 28 maio 2026.
- ISO/IEC. **ISO/IEC 27001:2022**: Information security, cybersecurity and privacy protection - Information security management systems - Requirements. Genebra: ISO, 2022.
- MEIRELLES, F. S. **Pesquisa Anual do FGVcia**. São Paulo: FGV, 2020.
- META. **WhatsApp Business Platform Cloud API**. Disponível em: <https://developers.facebook.com/docs/whatsapp>. Acesso em: 28 maio 2026.
- N8N. **n8n: Workflow automation for technical teams**. 2026. Disponível em: <https://n8n.io/>. Acesso em: 28 maio 2026.
- NIELSEN, Jakob. **Designing Web Usability: The Practice of Simplicity**. Indianapolis: New Riders Publishing, 2000.
- PRESSMAN, R. S. **Engenharia de Software**. 8. ed. Porto Alegre: AMGH, 2016.
- RIES, E. **A startup enxuta**: como os empreendedores atuais utilizam a inovação contínua para criar empresas extremamente bem-sucedidas. São Paulo: Leya, 2012.
- SCHWABER, K.; SUTHERLAND, J. **O Guia do Scrum**: as regras do jogo. Scrum.org, 2020. Disponível em: <https://scrumguides.org>. Acesso em: 28 maio 2026.
- SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2018.
- UNIVERSIDADE DE FRANCA (UNIFRAN). **Curso Superior de Tecnologia em Estética e Cosmética**. Franca: UNIFRAN, 2026. Disponível em: <https://cursos.unifran.edu.br/grad-estetica-e-cosmetica-unifran/p>. Acesso em: 28 maio 2026.
- AGENDAPRO. **Software de Gestão e Agendamento para Clínicas e Estética**. Disponível em: <https://agendapro.com/br>. Acesso em: 28 maio 2026.
- GETNINJAS. **Plataforma de Contratação de Serviços e Profissionais Autônomos**. Disponível em: <https://www.getninjas.com.br/moda-e-beleza/esteticista/quick-massage>. Acesso em: 28 maio 2026.
- ICLINIC. **Sistema de Gestão de Clínicas e Prontuário Eletrônico**. São Paulo: iClinic, 2026. Disponível em: <https://iclinic.com.br>. Acesso em: 28 maio 2026.
- ZENFISIO. **Sistema de Gestão para Clínicas e Consultórios de Fisioterapia e Estética**. Porto Alegre: ZenFisio, 2026. Disponível em: <https://zenfisio.com>. Acesso em: 28 maio 2026.